

C++ 프로그래밍

STL (Standard Template Library)

STL이란?

STL (Standard Template Library)은 C++98부터 표준에 포함된 **자료구조와 알고리즘 라이브러리**다.

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <algorithm>

int main() {
    std::vector<int> scores = {85, 92, 78, 95, 88};
    // 정렬 - 직접 구현 불필요
    std::sort(scores.begin(), scores.end());    // scores: {78, 85, 88, 92, 95}

    // 키-값 저장
    std::map<std::string, int> student;
    student["Alice"] = 95;
    student["Bob"]   = 88;
    std::cout << student["Alice"]; // 95
}
```

특성	내용
범용성	템플릿 기반 — 어떤 타입이든 동일한 인터페이스
표준 탑재	모든 C++ 컴파일러에 기본 포함
검증된 구현	최적화·안전성이 보장된 코드
재사용성	자료구조·알고리즘을 직접 구현할 필요 없음

STL의 4대 구성 요소

컨테이너 → 이터레이터 → 알고리즘 순서로 연계해 동작한다.

컨테이너 (Containers)

데이터를 **저장**하는 자료구조

```
std::vector<int> v = {1, 2, 3};    // 동적 배열
std::set<int> s = {3, 1, 2};      // 정렬된 집합
std::map<std::string, int> m;      // 키-값 저장소
m["one"] = 1;
```

이터레이터 (Iterators)

컨테이너를 **순회**하는 범용 포인터

```
std::vector<int> v = {10, 20, 30};

auto it = v.begin();    // 첫 원소를 가리킴
*it;                    // 10 - 역참조
++it;                   // 다음 원소로 이동
*it;                    // 20
```

STL의 4대 구성 요소

컨테이너 → 이터레이터 → 알고리즘 순서로 연계해 동작한다.

알고리즘 (Algorithms)

이터레이터 범위에 연산을 적용

```
#include <algorithm>
#include <numeric>

std::vector<int> v = {3, 1, 4, 1, 5};

std::sort(v.begin(), v.end());      // 정렬
// v: {1, 1, 3, 4, 5}

auto it = std::find(v.begin(), v.end(), 3);
// *it == 3

int sum = std::accumulate(v.begin(), v.end(), 0);
// sum == 14
```

함수 객체 / 람다 (Functors / Lambdas)

알고리즘에 동작을 주입하는 방식

```
// 내림차순 정렬 - 표준 함수 객체
std::sort(v.begin(), v.end(), std::greater<int>());

// 람다로 커스텀 조건 전달
auto it2 = std::find_if(v.begin(), v.end(),
    [](int x){ return x > 3; });
```

STL 헤더 파일

```
// — 시퀀스 컨테이너 —————
#include <vector>           // vector<T>           동적 배열 ← 기본 선택
#include <array>            // array<T, N>         고정 크기 배열 (C++11)
#include <list>             // list<T>            이중 연결 리스트
#include <deque>            // deque<T>           양방향 큐

// — 연관 컨테이너 (트리 기반, 키 순 정렬) —————
#include <set>              // set<T>, multiset<T>
#include <map>              // map<K, V>, multimap<K, V>

// — 비정렬 컨테이너 (해시 기반, C++11) —————
#include <unordered_set>    // unordered_set<T>
#include <unordered_map>    // unordered_map<K, V>

// — 알고리즘과 집계 —————
#include <algorithm>        // sort, find, transform, count, ...
#include <numeric>          // accumulate, iota, inner_product, ...
#include <functional>       // greater<T>, less<T>, std::function
```

기본 원칙: 특별한 이유가 없으면 `std::vector` 를 먼저 선택한다. 키 기반 검색이 필요하다면 `std::map` , 중복 없는 집합이면 `std::set` 을 고른다.

`std::vector`

동적 연속 메모리 공간의 시퀀스 컨테이너

std::vector — 생성과 기본 조작

vector 는 크기가 자동으로 늘어나는 동적 배열이다. STL에서 가장 자주 쓰는 컨테이너다.

생성

```
#include <vector>

// 빈 벡터
std::vector<int> v1;

// 초기값 리스트
std::vector<int> v2 = {10, 20, 30};

// N개를 같은 값으로 채우기
std::vector<int> v3(5, 0);    // {0, 0, 0, 0, 0}

// 복사 생성
std::vector<int> v4 = v2;    // {10, 20, 30}
```

뒤에 추가·제거, 기본 조회

```
std::vector<int> v;

v.push_back(10);    // v: {10}
v.push_back(20);    // v: {10, 20}
v.push_back(30);    // v: {10, 20, 30}

v.size();           // 3 - 원소 수
v.empty();          // false

v.front();           // 10 - 첫 번째 원소
v.back();            // 30 - 마지막 원소

v.pop_back();        // v: {10, 20}
```

std::vector + 알고리즘: 검색

<algorithm> 의 함수들은 **이터레이터 범위** (begin, end) 를 받아 동작한다. 컨테이너 종류와 무관하게 같은 인터페이스를 쓴다.

std::find — 값으로 검색

```
#include <algorithm>

std::vector<int> v = {10, 30, 20, 50, 40};

// 30을 검색 - 첫 번째 이터레이터 반환
auto it = std::find(v.begin(), v.end(), 30);

if (it != v.end()) {
    std::cout << *it;           // 30
    std::cout << (it - v.begin()); // 1 (인덱스)
} else {
    std::cout << "없음";
}

// 없는 값 검색 → end() 반환
auto it2 = std::find(v.begin(), v.end(), 99);
if (it2 == v.end()) {
    std::cout << "99는 없음";    // ← 실행됨
}
```

std::find_if — 조건으로 검색

```
std::vector<int> v = {10, 30, 20, 50, 40};

// 30보다 큰 첫 번째 원소
auto it = std::find_if(v.begin(), v.end(),
    [](int x){ return x > 30; });
// *it == 50

// 짝수인 첫 번째 원소
auto it2 = std::find_if(v.begin(), v.end(),
    [](int x){ return x % 2 == 0; });
// *it2 == 10

// 조건을 만족하는 원소가 하나라도 있는가?
bool any = std::any_of(v.begin(), v.end(),
    [](int x){ return x > 45; });
// true (50이 있음)
```

핵심 패턴: algorithm(v.begin(), v.end(), 조건) 조건이 필요하면 람다를 전달한다.

std::vector + 알고리즘: 정렬·집계

std::sort — 정렬

```
std::vector<int> v = {30, 10, 50, 20, 40};

// 오름차순 정렬 (기본)
std::sort(v.begin(), v.end());
// v: {10, 20, 30, 40, 50}

// 내림차순 — 비교 람다 전달
std::sort(v.begin(), v.end(),
    [](int a, int b){ return a > b; });
// v: {50, 40, 30, 20, 10}

// 부분 정렬 — 앞 3개만
std::sort(v.begin(), v.begin() + 3);
```

std::accumulate — 집계

```
#include <numeric>

std::vector<int> v = {10, 20, 30, 40, 50};

// 합계 (초기값 0)
int sum = std::accumulate(v.begin(), v.end(), 0);
// sum == 150

// 곱 (초기값 1, 커스텀 연산)
int product = std::accumulate(v.begin(), v.end(), 1,
    [](int acc, int x){ return acc * x; });
// product == 12'000'000
```

std::count_if — 조건에 맞는 개수

```
std::vector<int> v = {10, 20, 30, 40, 50};

int cnt = std::count_if(v.begin(), v.end(),
    [](int x){ return x >= 30; });
// cnt == 3 (30, 40, 50)
```

std::max_element / std::min_element

```
auto max_it = std::max_element(v.begin(), v.end());
auto min_it = std::min_element(v.begin(), v.end());

std::cout << *max_it;    // 50
std::cout << *min_it;    // 10
// (max_it - v.begin()) → 인덱스도 구할 수 있음
```

std::vector — 원소 접근과 수정

원소 접근

```
std::vector<int> v = {10, 20, 30, 40, 50};

// [] - 경계 검사 없음 (빠름)
v[0];      // 10
v[4];      // 50
// v[10];  // ❌ 미정의 동작

// at() - 경계 검사 있음
v.at(2);    // 30
try {
    v.at(10); // std::out_of_range 예외
} catch (const std::out_of_range& e) {
    std::cout << e.what();
}

v.front();  // 10
v.back();   // 50
```

값 변경

```
std::vector<int> v = {10, 20, 30, 40, 50};

// 인덱스로 직접 변경
v[1] = 99;
// v: {10, 99, 30, 40, 50}

// at()으로 변경 (경계 검사 포함)
v.at(3) = 77;
// v: {10, 99, 30, 77, 50}

// 범위 기반 for로 일괄 수정
for (auto& x : v) x *= 2;
// v: {20, 198, 60, 154, 100}

// assign: 전체를 새 값으로 교체
v.assign(3, 0); // v: {0, 0, 0}
```

std::vector — 삽입과 삭제

중간 삽입·삭제 시, 삽입·삭제 위치 뒤의 원소가 모두 이동 → $O(n)$. 중간 삽입·삭제가 잦다면 `std::list` 를 고려한다.

뒤에 추가·제거 — $O(1)$ amortized

```
std::vector<int> v = {10, 20, 30};

// push_back: 값을 복사·이동해 뒤에 추가
v.push_back(40);      // {10, 20, 30, 40}

// emplace_back: 인자를 받아 제자리 생성 (더 효율적)
v.emplace_back(50);    // {10, 20, 30, 40, 50}

// pop_back: 마지막 원소 제거
v.pop_back();          // {10, 20, 30, 40}
```

중간·앞에 삽입 — $O(n)$

```
std::vector<int> v = {10, 30, 40};

// insert(위치, 값) - 위치 앞에 삽입
v.insert(v.begin() + 1, 20);
// v: {10, 20, 30, 40}

// 여러 개 한 번에 삽입
v.insert(v.end(), {50, 60});
// v: {10, 20, 30, 40, 50, 60}
```

삭제 — $O(n)$ (뒤 제거는 $O(1)$)

```
std::vector<int> v = {10, 20, 30, 40, 50};

// 이터레이터 위치 삭제 - 다음 이터레이터 반환
auto it = v.begin() + 2;
it = v.erase(it);      // 30 삭제
// v: {10, 20, 40, 50}, *it == 40

// 범위 삭제 [first, last)
v.erase(v.begin() + 1, v.begin() + 3);
// v: {10, 50}

// 전체 삭제 (capacity는 유지됨)
v.clear();
```

왜 중간 삽입·삭제가 $O(n)$ 인가?

```
insert(begin+1, 20):
[ 10 | 30 | 40 ] → [ 10 | 20 | 30 | 40 ]
                 30, 40이 한 칸씩 오른쪽으로 이동

erase(begin+2):
[ 10 | 20 | 30 | 40 | 50 ] → [ 10 | 20 | 40 | 50 ]
                          40, 50이 한 칸씩 왼쪽으로 이동
```

std::vector — size와 capacity

vector 는 내부에 **동적 배열**을 갖는다. 공간이 부족하면 더 큰 배열로 **재할당**한다.

size vs capacity

```
std::vector<int> v;

v.size();      // 0 - 현재 원소 수
v.capacity();  // 0 - 할당된 공간

v.push_back(1);
v.push_back(2);
v.push_back(3);
v.size();      // 3
v.capacity();  // 4 (구현마다 다름, 보통 2배씩 확장)

v.push_back(4);
v.push_back(5); // 용량 초과 → 재할당
v.capacity();   // 8
```

push_back 시 capacity 초과 → 재할당

- ① 더 큰 배열 할당
- ② 기존 원소 전부 복사
- ③ 기존 배열 해제

reserve 로 재할당 방지

```
// ❌ push_back마다 재할당 가능
std::vector<int> v1;
for (int i = 0; i < 1000; i++)
    v1.push_back(i); // 재할당: 1→2→4→8→...

// ✅ 미리 공간 확보 → 재할당 0회
std::vector<int> v2;
v2.reserve(1000);
for (int i = 0; i < 1000; i++)
    v2.push_back(i);
```

resize — 크기 직접 조정

```
std::vector<int> v = {1, 2, 3, 4, 5};

v.resize(3);           // v: {1, 2, 3} (뒤가 잘림)
v.resize(6, 0);        // v: {1, 2, 3, 0, 0, 0}
v.clear();              // v: {} (size=0, capacity 유지)
```

원소 수를 미리 알면 reserve() 로 재할당을 방지해 성능을 높인다.

```
std::list
```

이중 연결 리스트 구조의 시퀀스 컨테이너

std::list — 구조와 생성

list 는 각 원소가 앞뒤 원소를 가리키는 포인터를 갖는 이중 연결 리스트다. 메모리가 불연속이다.

메모리 구조

```
1  vector  [ 10 | 20 | 30 | 40 ]  연속 메모리
2          ↑                      한 블록
3
4  list    [·|10|·] ↔ [·|20|·] ↔ [·|30|·] ↔ [·|40|·]
5          prev next  prev next  prev next  prev next
6                      각 노드 개별 힙 할당
```

생성

```
#include <list>

std::list<int> lst1;           // 빈 리스트
std::list<int> lst2 = {10, 20, 30}; // 초기화
std::list<int> lst3(4, 0);     // {0, 0, 0, 0}
```

앞·뒤 추가·제거와 기본 조회

```
std::list<int> lst = {20, 30};

lst.push_front(10); // {10, 20, 30}
lst.push_back(40);  // {10, 20, 30, 40}

lst.pop_front();    // {20, 30, 40}
lst.pop_back();     // {20, 30}

lst.front();        // 20
lst.back();         // 30
lst.size();         // 2
lst.empty();        // false
```

push_front / pop_front 는 vector 에 없는 기능이다. 앞·뒤 모두 $O(1)$ 이다.

std::list — 삽입·삭제: O(1)

list 의 핵심 장점 — 이터레이터가 가리키는 위치에 삽입·삭제가 **O(1)**이다. 포인터만 변경하고 원소 이동은 없다.

insert — O(1) 중간 삽입

```
std::list<int> lst = {10, 30, 40};

auto it = std::find(lst.begin(), lst.end(), 30);

// it 앞에 20 삽입 - O(1)
lst.insert(it, 20);
// lst: {10, 20, 30, 40}

// 삽입 전: ... ↔ [10] ↔ [30] ↔ ...
// 삽입 후: ... ↔ [10] ↔ [20] ↔ [30] ↔ ...
// (포인터 4개만 수정, 원소 이동 없음)
```

erase · remove · remove_if

```
std::list<int> lst = {10, 20, 30, 20, 40};

// 이터레이터 위치 삭제 - O(1)
auto it = std::find(lst.begin(), lst.end(), 30);
auto next = lst.erase(it); // 30 삭제
// lst: {10, 20, 20, 40}, *next == 20

// 값으로 전체 삭제
lst.remove(20);
// lst: {10, 40}

// 조건으로 전체 삭제
lst.remove_if([](int x){ return x > 15; });
// lst: {10}
```

vector와 비교:

```
std::vector<int> v = {10, 30, 40};
auto it2 = std::find(v.begin(), v.end(), 30);

// 삽입 - O(n): 30, 40이 오른쪽으로 이동
v.insert(it2, 20);
// [10] [20←30] [30←40] [40←?] 원소 전체 시프트
```

연산

	vector	list
중간 삽입	O(n) 이동	O(1) 포인터
중간 삭제	O(n) 이동	O(1) 포인터
삽입 후 기존 이터레이터	❌ 무효화	✅ 유지

std::list — 멤버 알고리즘

list 는 구조 특성을 활용한 **전용 멤버 함수**를 제공한다. 포인터만 바꾸기 때문에 원소 복사 없이 동작한다.

sort / reverse / unique

```
std::list<int> lst = {30, 10, 40, 20, 10, 30};

// 오름차순 정렬 (std::sort 대신)
lst.sort();
// lst: {10, 10, 20, 30, 30, 40}

// 내림차순
lst.sort([](int a, int b){ return a > b; });
// lst: {40, 30, 30, 20, 10, 10}

// 역순으로 뒤집기
lst.reverse();
// lst: {10, 10, 20, 30, 30, 40}

// 정렬 후 연속 중복 제거
lst.sort();
lst.unique();
// lst: {10, 20, 30, 40}
```

splice — O(1) 구간 이동

다른 리스트의 원소를 **포인터 연결만 바꿔** 이동한다. 원소 복사가 없다.

```
std::list<int> a = {1, 2, 3, 4, 5};
std::list<int> b = {10, 20, 30};

// b 전체를 a의 3 앞으로 이동 - O(1)
auto pos = std::find(a.begin(), a.end(), 3);
a.splice(pos, b);
// a: {1, 2, 10, 20, 30, 3, 4, 5}
// b: {} (b는 비워짐)

// b의 원소 하나만 이동
std::list<int> c = {100, 200, 300};
auto from = std::find(c.begin(), c.end(), 200);
a.splice(a.begin(), c, from);
// a: {200, 1, 2, 10, 20, 30, 3, 4, 5}
```

splice 는 vector 에 없는 list 고유 기능이다. 노드 포인터만 재연결 — **복사·이동 없이 O(1)**.


```
std::array
```

정적 연속 메모리 공간의 시퀀스 컨테이너

std::array — 고정 크기 배열

컴파일 타임에 크기가 고정된 배열. C 배열과 성능은 같고, STL 인터페이스를 추가로 제공한다.

```
#include <array>

////////////////////
// 생성과 원소 접근
////////////////////
std::array<int, 5> a = {1, 2, 3, 4, 5};

a[0];          // 1  -  경계 검사 없음 (빠름)      a.front(); // 1
a.at(2);        // 3  -  경계 검사 있음 (안전)      a.back();   // 5
a.size();        // 5  -  항상 고정                a.empty(); // false

for (const auto& x : a) { std::cout << x << " "; } // 1 2 3 4 5
std::sort(a.begin(), a.end()); // STL 알고리즘 그대로 사용 가능

////////////////////
// 함수 파라미터로 전달
////////////////////

// ✅ std::array - 크기가 타입에 포함된 채로 전달
void print(const std::array<int, 5>& a) {
    for (const auto& x : a) std::cout << x << " ";
}

std::array<int, 5> a5 = {1, 2, 3, 4, 5};
print(a5); // ✅ array<int, 5> - 타입 일치
```

std::array — fill / swap / data

fill / swap — 일괄 조작

```
std::array<int, 5> a = {1, 2, 3, 4, 5};
std::array<int, 5> b = {6, 7, 8, 9, 10};

// fill: 전체를 같은 값으로 채우기
a.fill(0);
// a: {0, 0, 0, 0, 0}

// swap: 두 배열 내용 교환 - O(N)
a.swap(b);
// a: {6, 7, 8, 9, 10}, b: {0, 0, 0, 0, 0}

// 값 타입 - 복사·대입 가능
std::array<int, 5> c = a; // 복사 생성
b = c; // 대입
// (C 배열은 둘 다 불가)
```

data() — C API 호환

```
std::array<int, 5> a = {1, 2, 3, 4, 5};

// data(): 첫 원소의 raw 포인터 반환
int* p = a.data();

// C 스타일 함수에 그대로 전달 가능
void legacy_print(const int* arr, int n);
legacy_print(a.data(), a.size());

// 2차원 배열
std::array<std::array<int, 3>, 2> matrix = {{
    {1, 2, 3},
    {4, 5, 6}
}};
matrix[0][1]; // 2
matrix[1][2]; // 6
for (const auto& row : matrix)
    for (int x : row)
        std::cout << x << " ";
```

data() 로 raw 포인터를 얻을 수 있어 C 라이브러리와 **호환성**을 유지하면서 STL 인터페이스도 사용할 수 있다.

시퀀스 컨테이너 선택 가이드

```
1  순서 있는 데이터를 저장해야 한다
2
3  └─ 크기가 컴파일 타임에 고정인가?
4      → std::array<T, N>
5
6  └─ 끝에서만 추가·삭제하는가?                ← 대부분의 경우
7      → std::vector<T>                        ← 기본 선택
8
9  └─ 중간 삽입·삭제가 잦고, 이터레이터를 유지해야 하는가?
10     → std::list<T>
11     (임의 접근 없음, 캐시 효율 낮음 - 실제 성능은 측정 필요)
```

컨테이너	이터레이터	임의 접근	앞 삽입	중간 삽입	삽입 후 이터레이터
<code>vector</code>	Random Access	$O(1)$ ✓	$O(n)$	$O(n)$	✗ 무효화
<code>array</code>	Random Access	$O(1)$ ✓	—	—	—
<code>list</code>	Bidirectional	✗	$O(1)$ ✓	$O(1)$ ✓	✓ 유지

결론: 이유 없이 `vector` 를 써도 거의 틀리지 않는다. 삽입·삭제 중 이터레이터를 살려야 한다면 `list` 를 고려한다.

C++ 프로그래밍

이터레이터 (Iterators)

순회 문제 — 컨테이너마다 다른가?

`vector` 와 `list` 는 내부 구조가 완전히 다르다. 그런데 어떻게 **같은 방식**으로 순회할 수 있을까?

`std::vector` 순회

```
std::vector<int> v = {10, 20, 30, 40, 50};

// 이터레이터 for 루프
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";    // 10 20 30 40 50
}

// 범위 기반 for (내부적으로 begin/end 호출)
for (const auto& x : v) {
    std::cout << x << " ";    // 10 20 30 40 50
}
```

`std::list` 순회

```
std::list<int> lst = {10, 20, 30, 40, 50};

// 문법이 vector와 완전히 동일
for (auto it = lst.begin(); it != lst.end(); ++it) {
    std::cout << *it << " ";    // 10 20 30 40 50
}

// 범위 기반 for
for (const auto& x : lst) {
    std::cout << x << " ";    // 10 20 30 40 50
}
```

내부 구조는 달라도 순회 코드는 동일 — 이것이 이터레이터 덕분이다.

이터레이터란?

이터레이터는 컨테이너의 원소를 가리키는 객체다. 포인터와 유사한 인터페이스를 제공하며, 컨테이너 종류와 무관하게 동일한 방식으로 순회할 수 있다.

포인터처럼 사용

```
std::vector<int> v = {10, 20, 30, 40, 50};

// 이터레이터 선언
std::vector<int>::iterator it = v.begin();

*it;    // 10 - 역참조
++it;   // 다음으로 이동
*it;    // 20

it += 2; // 2칸 앞으로 (random access)
*it;     // 40

// auto로 간결하게
auto it2 = v.begin();
std::cout << *it2; // 10
```

begin / end

```
std::vector<int> v = {10, 20, 30};

// begin(): 첫 번째 원소를 가리킴
// end():   마지막 다음을 가리킴 (유효한 원소 아님)

auto first = v.begin(); // → 10
auto last  = v.end();   // → (30 다음, 접근 불가)

// 반복 패턴
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " "; // 10 20 30
}
```

end() 는 마지막 원소의 다음 위치를 가리키며, 역참조(*end())는 미정의 동작이다.

```
begin()           end()
  ↓               ↓
[ 10 | 20 | 30 | × ]
```

end() — 이터레이터의 nullptr

포인터에서 "유효하지 않음"을 nullptr 로 표현하듯, 이터레이터에서 "원소가 없음 / 범위의 끝"을 end() 로 표현함.

포인터의 nullptr vs 이터레이터의 end()

```
// — 포인터 패턴 —————
int* find_ptr(int* arr, int n, int val) {
    for (int i = 0; i < n; i++)
        if (arr[i] == val) return &arr[i];
    return nullptr; // 못 찾으면 nullptr 반환
}

int arr[] = {10, 20, 30};
int* p = find_ptr(arr, 3, 99);
if (p != nullptr) { // 항상 확인 후 사용
    std::cout << *p;
}

// — 이터레이터 패턴 —————
std::vector<int> v = {10, 20, 30};
auto it = std::find(v.begin(), v.end(), 99);
if (it != v.end()) { // end() == nullptr 역할
    std::cout << *it;
}
```

end() 반환 패턴 — vector / list

```
std::vector<int> v = {10, 20, 30};
std::list<int> lst = {10, 20, 30};

// ① vector: 값이 없으면 end() 반환
auto it1 = std::find(v.begin(), v.end(), 99);
if (it1 == v.end()) std::cout << "없음\n";

// ② vector: 조건 불만족 시 end() 반환
auto it2 = std::find_if(v.begin(), v.end(),
    [](int x){ return x > 100; });
if (it2 == v.end()) std::cout << "조건 불만족\n";

// ③ list도 동일한 패턴
auto it3 = std::find(lst.begin(), lst.end(), 99);
if (it3 == lst.end()) std::cout << "없음\n";

// ④ list: 조건으로 검색
auto it4 = std::find_if(lst.begin(), lst.end(),
    [](int x){ return x > 100; });
if (it4 == lst.end()) std::cout << "조건 불만족\n";
```

⚠ end() 를 역참조하면 **미정의 동작(크래시)**. 이터레이터는 반드시
!= end() **확인 후** 역참조한다.

	포인터	이터레이터
"없음" 표현	nullptr	end()
역참조 전 확인	!= nullptr	!= end()

이터레이터 카테고리

이터레이터는 지원하는 연산에 따라 5가지 카테고리로 나뉜다. 상위로 갈수록 더 많은 연산을 지원한다.

카테고리	지원 연산	대표 컨테이너
Input	<code>*</code> , <code>++</code> , <code>==</code> , <code>!=</code> (읽기 전용, 1회)	<code>istream_iterator</code>
Forward	Input + 다중 패스	<code>forward_list</code>
Bidirectional	Forward + <code>--</code> (역방향)	<code>list</code> , <code>set</code> , <code>map</code>
Random Access	Bidirectional + <code>+n</code> , <code>-n</code> , <code>[]</code> , <code><</code>	<code>vector</code> , <code>array</code> , <code>deque</code>
Output	<code>*</code> , <code>++</code> (쓰기 전용)	<code>ostream_iterator</code>

```
std::vector<int> v = {1, 2, 3, 4, 5};

auto it = v.begin();           // random access iterator
it += 3;                       // ✅ vector: 3칸 이동 가능
it[1];                         // ✅ 인덱스 접근 가능

std::list<int> lst = {1, 2, 3, 4, 5};
auto it2 = lst.begin();        // bidirectional iterator
++it2;                         // ✅
--it2;                         // ✅
// it2 += 3;                   // ❌ list는 random access 불가
```

range-based for 와 auto

C++11부터 범위 기반 for 루프로 이터레이터를 명시하지 않고도 순회할 수 있다.

범위 기반 for

```
std::vector<int> v = {10, 20, 30};

// 값으로 복사 - 원본 변경 안 됨
for (int x : v) {
    std::cout << x << " "; // 10 20 30
}

// 참조로 순회 - 원본 수정 가능
for (int& x : v) {
    x *= 2;
}
// v: {20, 40, 60}

// const 참조 - 읽기 전용, 복사 비용 없음 (권장)
for (const int& x : v) {
    std::cout << x << " "; // 20 40 60
}

// auto로 간결하게
for (const auto& x : v) {
    std::cout << x << " ";
}
```

list / array / string에도 적용 가능

```
// list - vector와 완전히 동일한 구문
std::list<int> lst = {10, 20, 30};
for (const auto& x : lst) {
    std::cout << x << " "; // 10 20 30
}

// array
std::array<int, 3> arr = {1, 2, 3};
for (const auto& x : arr) {
    std::cout << x << " "; // 1 2 3
}

// string (char 단위 순회)
std::string s = "hello";
for (char c : s) {
    std::cout << c << " "; // h e l l o
}
```

범위 기반 for는 내부적으로 `begin()` / `end()` 를 호출한다. `set`, `map` 등 연관 컨테이너도 같은 구문 — 연관 컨테이너 단원에서 다룬다.

const_iterator / reverse_iterator

함수	반환 타입	방향	수정
<code>begin()</code> / <code>end()</code>	<code>iterator</code>	정방향	✓
<code>cbegin()</code> / <code>cend()</code>	<code>const_iterator</code>	정방향	✗
<code>rbegin()</code> / <code>rend()</code>	<code>reverse_iterator</code>	역방향	✓
<code>crbegin()</code> / <code>crend()</code>	<code>const_reverse_iterator</code>	역방향	✗

const_iterator — 읽기 전용

```
std::vector<int> v = {10, 20, 30};

// const_iterator: 원소 수정 불가
std::vector<int>::const_iterator cit = v.cbegin();
std::cout << *cit;    // 10
// *cit = 99;    // ✗ 컴파일 오류 - 읽기 전용

// auto 사용 (cbegin/cend 권장)
for (auto it = v.cbegin(); it != v.cend(); ++it) {
    std::cout << *it << " ";    // 10 20 30
}

// const 컨테이너 - 자동으로 const_iterator
const std::vector<int> cv = {1, 2, 3};
for (auto it = cv.begin(); it != cv.end(); ++it) {
    // it는 자동으로 const_iterator
}
```

reverse_iterator — 역방향 순회

```
std::vector<int> v = {10, 20, 30, 40, 50};

// rbegin(): 마지막 원소를 가리킴
// rend(): 첫 번째 이전을 가리킴
for (auto it = v.rbegin(); it != v.rend(); ++it) {
    std::cout << *it << " ";
    // 50 40 30 20 10
}

// list도 역방향 순회 가능
std::list<int> lst = {10, 20, 30};
for (auto it = lst.rbegin(); it != lst.rend(); ++it) {
    std::cout << *it << " ";    // 30 20 10
}
```

C++ 프로그래밍

연관 컨테이너 (Associative Containers)

```
std::set
```

중복을 허용하지 않는 원소(Key)들의 연관 컨테이너

std::set — 중복 없는 집합

set 은 중복을 허용하지 않는 집합이다. 원소의 삽입·검색·삭제를 빠르게 수행할 수 있다.

기본 사용

```
#include <set>

std::set<int> s;

// 삽입 — 중복 무시
s.insert(30);
s.insert(10);
s.insert(20);
s.insert(10);    // 중복 — 무시됨

// 순회
for (int x : s) {
    std::cout << x << " "; // 10 20 30
}

s.size();    // 3
s.empty();   // false
```

검색과 삭제

```
std::set<int> s = {10, 20, 30, 40, 50};

// 검색 — O(log n)
auto it = s.find(30);
if (it != s.end()) {
    std::cout << *it;    // 30
}

// 존재 여부 확인
s.count(30);    // 1 (있음) 또는 0 (없음)
// s.contains(30);    // C++20 ✔️

// 삭제
s.erase(20);    // s: {10, 30, 40, 50}

// 범위 삭제 — [30, 50)
s.erase(s.find(30), s.find(50));
// s: {10, 50}
```

find() 는 $O(\log n)$ — std::find() (선형 탐색)보다 훨씬 빠르다.

std::set — 초기화와 활용 패턴

다양한 초기화

```
// initializer list - 중복 자동 제거
std::set<int> s1 = {5, 3, 1, 4, 2, 3, 1};
// s1: {1, 2, 3, 4, 5}

// 문자열 set
std::set<std::string> words;
words.insert("banana");
words.insert("apple");
words.insert("cherry");
words.insert("apple"); // 중복 무시
// words: {"apple", "banana", "cherry"}

// vector → set: 중복 제거
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6, 5};
std::set<int> s2(v.begin(), v.end());
// s2: {1, 2, 3, 4, 5, 6, 9}
```

활용 패턴

```
// ① 방문 여부 추적
std::set<int> visited;
visited.insert(3);
visited.insert(5);
visited.insert(3); // 이미 방문 - 무시

if (visited.count(5)) {
    std::cout << "5는 이미 방문\n"; // ✅
}

// ② 중복 제거 후 vector로 복원
std::vector<int> data = {3, 1, 4, 1, 5, 9, 2, 6, 5};
std::set<int> s(data.begin(), data.end());
std::vector<int> unique_data(s.begin(), s.end());
// unique_data: {1, 2, 3, 4, 5, 6, 9}

// ③ insert 반환값 - 삽입 성공 여부 확인
auto [it, ok] = s1.insert(99);
// ok == true → 삽입됨
// ok == false → 이미 존재
```

set 은 **중복 없는 원소 집합**이 필요할 때 사용한다. 방문 추적, 중복 제거, 멤버십 확인에 자주 활용된다.

`std::map`

Key-Value 쌍의 연관 컨테이너

std::map — 키로 값을 찾는 사전

map 은 키(key)로 값(value)을 찾는 사전이다. 배열이 정수 인덱스로 값을 찾듯, map 은 임의 타입의 키로 값을 찾는다.

배열 vs map

vector<int>

인덱스(int) → 값

0 → 95
1 → 88
2 → 72

// vector: 정수 인덱스만 가능

```
std::vector<int> v = {95, 88, 72};  
v[0]; // 95
```

// map: 문자열·정수·객체 등 키로 사용 가능

```
std::map<std::string, int> score;  
score["Alice"] = 95;  
score["Alice"]; // 95 - 이름으로 직접 접근
```

map<string, int>

키(string) → 값

"Alice" → 95
"Bob" → 88
"Carol" → 72

활용 예시

```
// 영한 사전: 단어 → 뜻  
std::map<std::string, std::string> dict;  
dict["apple"] = "사과";  
dict["banana"] = "바나나";  
dict["apple"]; // "사과"
```

```
// 학번 → 점수  
std::map<int, double> grade;  
grade[20241001] = 4.5;  
grade[20241002] = 3.8;
```

```
// 단어 빈도 카운팅  
std::map<std::string, int> freq;  
freq["hello"]++;  
freq["world"]++;  
freq["hello"]++;  
// freq["hello"] == 2
```

- 키는 어떤 타입이든 가능 (단, < 비교 가능해야 함)
- 삽입·검색·삭제 모두 $O(\log n)$

std::map — 생성과 삽입

map 은 **키-값 쌍**을 저장한다. 키는 중복될 수 없으며, 빠르게 삽입·검색·삭제할 수 있다.

생성

```
#include <map>

// 빈 map
std::map<std::string, int> m1;

// initializer list
std::map<std::string, int> score = {
    {"Alice", 95},
    {"Bob", 88},
    {"Carol", 72}
};
```

insert — 명시적 삽입

```
std::map<std::string, int> m;

m.insert({"Dave", 85});

// insert는 이미 있는 키를 덮어쓰지 않는다
auto [it, ok] = m.insert({"Dave", 99}); // C++17
// ok == false - 삽입 실패 (Dave 이미 존재)
// it → 기존 Dave 원소를 가리킴
```

emplace / try_emplace / insert_or_assign

```
std::map<std::string, int> m;

// emplace: 키-값을 전달해 제자리 생성
m.emplace("Alice", 95);

// try_emplace (C++17): 없을 때만 삽입
m.try_emplace("Alice", 0); // 이미 있으면 무시
m.try_emplace("Bob", 88); // 없으면 삽입

// insert_or_assign (C++17): 항상 갱신
m.insert_or_assign("Alice", 100); // 있으면 덮어씀
```

메서드	이미 있는 키	없는 키
insert	무시	삽입
try_emplace	무시	삽입
insert_or_assign	갱신	삽입
[]	갱신	0으로 삽입

std::map — [] 연산자: 편리함과 함정

[] 로 삽입.갱신

```
std::map<std::string, int> m;

// [] 로 삽입
m["Alice"] = 95;
m["Bob"]    = 88;

// [] 로 접근
std::cout << m["Alice"]; // 95

// [] 로 수정 (덮어씀)
m["Alice"] = 100;
```

⚠️ 없는 키에 접근하면 자동 삽입!

```
std::map<std::string, int> m;
m["Alice"] = 95;

// 없는 키를 읽으면 0으로 초기화 후 삽입
std::cout << m["Nobody"]; // 0 출력
std::cout << m.size();    // 2 ← Nobody가 추가됨!

// const map에서는 [] 사용 불가
const auto& cm = m;
// cm["Alice"]; // ❌ 컴파일 오류
```

✅ 안전한 읽기: at() / find()

```
std::map<std::string, int> m = {
    {"Alice", 95}, {"Bob", 88}
};

// at() - 없으면 std::out_of_range 예외
std::cout << m.at("Alice"); // 95
try {
    m.at("Nobody"); // ❌ 예외 발생
} catch (const std::out_of_range&) {
    std::cout << "키 없음\n";
}

// find() - 없으면 end() 반환 (가장 안전)
auto it = m.find("Bob");
if (it != m.end()) {
    std::cout << it->second; // 88
}

// C++20: contains()
if (m.contains("Alice")) {
    std::cout << m.at("Alice");
}
```

원칙: [] 는 쓰기 전용, 읽기에는 at() 또는 find() 를 사용한다.

std::map — 검색.범위 조회

find / count / contains

```
std::map<std::string, int> score = {
    {"Alice", 95}, {"Bob", 88},
    {"Carol", 72}, {"Dave", 85}
};

// find: O(log n), 이터레이터 반환
auto it = score.find("Bob");
if (it != score.end()) {
    it->first;      // "Bob" - 키
    it->second;     // 88 - 값
    it->second = 90; // 값 수정도 가능
}

// count: 키 존재 여부 (0 또는 1)
score.count("Alice"); // 1
score.count("Nobody"); // 0

// C++20: contains
score.contains("Carol"); // true
```

lower_bound / upper_bound — 키 범위 조회

```
std::map<int, std::string> m = {
    {10, "A"}, {20, "B"}, {30, "C"}, {40, "D"}
};

// lower_bound: 키 이상인 첫 번째 이터레이터
auto lb = m.lower_bound(20);
// lb → {20, "B"}

// upper_bound: 키 초과인 첫 번째 이터레이터
auto ub = m.upper_bound(30);
// ub → {40, "D"}

// 키 [20, 30] 범위 순회
for (auto it = m.lower_bound(20);
     it != m.upper_bound(30); ++it) {
    std::cout << it->first << " "; // 20 30
}
```

lower_bound / upper_bound 모두 $O(\log n)$. 특정 키 범위를 빠르게 조회하는 것은 map 의 고유 강점이다.

std::map — 이터레이터와 순회

it->first / it->second

```
std::map<std::string, int> score = {
    {"Alice", 95}, {"Bob", 88}, {"Carol", 72}
};

// 이터레이터 순회
for (auto it = score.begin(); it != score.end(); ++it) {
    std::cout << it->first          // 키
                << ": " << it->second // 값
                << "\n";
}
// Alice: 95
// Bob: 88
// Carol: 72

// 역방향 순회
for (auto it = score.rbegin(); it != score.rend(); ++it) {
    std::cout << it->first << "\n";
    // Carol → Bob → Alice
}
```

범위 기반 for / 구조화 바인딩

```
// C++11: pair로 순회
for (const auto& p : score) {
    std::cout << p.first << ": " << p.second << "\n";
}

// C++17: structured bindings ← 권장
for (const auto& [name, s] : score) {
    std::cout << name << ": " << s << "\n";
}
// Alice: 95
// Bob: 88
// Carol: 72

// 값 수정: 참조로 받기
for (auto& [name, s] : score) {
    s += 5; // 모든 점수 +5
}
```

map은 항상 **일관된 키 순서**로 순회된다. C++17 구조화 바인딩으로
it->first / it->second 없이 읽기 쉽게 쓸 수 있다.

std::map — 삭제와 활용 패턴

삭제

```
std::map<std::string, int> m = {
    {"Alice", 95}, {"Bob", 88}, {"Carol", 72}
};

// 키로 삭제 - O(log n)
m.erase("Bob");
// m: {Alice:95, Carol:72}

// 이터레이터로 삭제
auto it = m.find("Carol");
if (it != m.end()) m.erase(it);
// m: {Alice:95}

// 범위 삭제 [first, last)
m.erase(m.begin(), m.end()); // == m.clear()

m.size(); // 0
m.empty(); // true
```

활용 패턴: 빈도 카운팅

[] 의 "없으면 0 삽입" 특성을 활용한 전형적인 패턴이다.

```
std::vector<std::string> words = {
    "apple", "banana", "apple",
    "cherry", "banana", "apple"
};

std::map<std::string, int> freq;

for (const auto& w : words) {
    freq[w]++; // 없으면 0으로 삽입 후 +1
}
// freq: {apple:3, banana:2, cherry:1}

for (const auto& [word, cnt] : freq) {
    std::cout << word << ": " << cnt << "\n";
}

// 가장 많이 등장한 단어
auto max_it = std::max_element(freq.begin(), freq.end(),
    [](const auto& a, const auto& b){
        return a.second < b.second;
    });
std::cout << max_it->first; // "apple"
```

std::pair — 두 값을 묶는 타입

map 의 각 원소는 std::pair<const Key, Value> 로 저장됨. pair 를 이해하면 map 이터레이터를 자연스럽게 읽을 수 있음.

기본 사용

```
#include <utility>    // std::pair

// 선언과 초기화
std::pair<std::string, int> p1 {"Alice", 95};

// make_pair - 타입 자동 추론
auto p2 = std::make_pair("Bob", 88);

// 중괄호 초기화 (C++11)
std::pair<std::string, int> p3 = {"Carol", 72};

// 멤버 접근
p1.first;    // "Alice" - 첫 번째 값
p1.second;   // 95      - 두 번째 값

// 비교 - first 기준, 같으면 second 기준
p1 < p3;     // "Alice" < "Carol" → true
```

map 의 원소 타입

```
// map<K, V>의 value_type = pair<const K, V>

std::map<std::string, int> score = {
    {"Alice", 95}, {"Bob", 88}    // 각 원소가 pair
};

// 이터레이터 → pair를 가리킴
auto it = score.begin();
it->first;    // "Alice" - 키 (const, 수정 불가)
it->second;   // 95      - 값

// insert에 pair 전달
score.insert(std::make_pair("Dave", 85));
score.insert({"Eve", 79});    // 중괄호 초기화도 가능

// range-based for: pair로 받기
for (const auto& p : score) {
    std::cout << p.first << ": " << p.second << "\n";
}
// → C++17 구조화 바인딩으로 더 간결하게 사용 가능
```

map 이터레이터의 ->first / ->second 는 pair<const Key, Value> 의 두 멤버다.

연관 컨테이너 비교

컨테이너	중복 키	[]	헤더	주 용도
<code>set<T></code>	✗	✗	<code><set></code>	중복 없는 집합, 존재 여부 확인
<code>map<K,V></code>	✗	✓	<code><map></code>	키-값 저장, 빠른 검색

```
1  연관 컨테이너가 필요한 경우
2
3  └─ 중복 없는 집합이 필요한가?
4      → set<T>   (예: 방문한 노드 추적, 단어 사전)
5
6  └─ 키-값 쌍을 저장해야 하는가?
7      → map<K,V> (예: 이름→점수, 단어 빈도 카운팅)
```

모든 연관 컨테이너는 삽입·검색·삭제가 $O(\log n)$ 이다. 순서가 필요 없고 $O(1)$ 검색이 중요하다면 `unordered_map` / `unordered_set` 을 고려한다.

C++ 프로그래밍

STL 알고리즘 (Algorithms)

알고리즘 — 컨테이너에 독립적인 연산

STL 알고리즘은 **이터레이터 범위**를 받아 동작한다. 컨테이너 종류와 무관하게 동일한 알고리즘을 적용할 수 있다.

```
#include <algorithm>    // sort, find, transform, count, ...
#include <numeric>       // accumulate, iota, ...

std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
std::list<int> l = {3, 1, 4, 1, 5, 9, 2, 6};

// 같은 알고리즘을 다른 컨테이너에 적용
std::sort(v.begin(), v.end());    // ✅ vector - random access
std::sort(l.begin(), l.end());    // ❌ list - bidirectional only
l.sort();                        // ✅ list 전용 멤버 함수
```

분류	대표 함수	헤더
검색	find, find_if, count, count_if, any_of, all_of	<algorithm>
정렬	sort, stable_sort, partial_sort, nth_element	<algorithm>
변환	transform, copy, copy_if, fill, generate	<algorithm>
집계	accumulate, max_element, min_element	<numeric>
집합	set_union, set_intersection, set_difference	<algorithm>

커스텀 함수 객체 (Functor)

`operator()` 를 정의한 클래스를 **함수 객체(function object)**라 한다. 인스턴스를 만들어 함수처럼 호출하거나, STL 알고리즘에 직접 전달할 수 있다.

Functor 정의

```
// 상태 없는 functor
struct IsEven {
    bool operator()(int x) const { return x % 2 == 0; }
};

// 생성자로 상태를 주입하는 functor
struct GreaterThan {
    int threshold;
    GreaterThan(int t) : threshold(t) {}
    bool operator()(int x) const { return x > threshold; }
};
```

인스턴스 생성과 활용

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};

// ① 인스턴스 생성 후 직접 호출
IsEven isEven;
isEven(4); // true - 일반 함수처럼 호출
isEven(3); // false

// ② 인스턴스를 변수에 담아 알고리즘에 전달
auto cnt = std::count_if(v.begin(), v.end(), isEven);
// cnt == 3

// ③ 상태를 주입한 인스턴스 생성 후 전달
GreaterThan gt3(3); // threshold = 3
gt3(5); // true
gt3(2); // false

auto it = std::find_if(v.begin(), v.end(), gt3);
// *it == 4

// ④ 임시 객체로 바로 전달 (①②③ 축약)
std::count_if(v.begin(), v.end(), IsEven{});
std::find_if(v.begin(), v.end(), GreaterThan{3});
```

표준 라이브러리 Functor (<functional>)

<functional> 헤더는 자주 쓰는 연산을 functor로 미리 정의해 제공. 직접 만들 필요 없이 알고리즘에 바로 전달 가능.

알고리즘에 활용

```
#include <functional>

std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

// std::greater<T>: 내림차순 정렬
std::sort(v.begin(), v.end(), std::greater<int>());
// v: {9, 6, 5, 4, 3, 2, 1, 1}

// std::plus<T>: accumulate 덧셈
int sum = std::accumulate(v.begin(), v.end(), 0,
                          std::plus<int>());
// sum == 31

// std::negate<T>: 부호 반전
std::vector<int> neg(v.size());
std::transform(v.begin(), v.end(),
               neg.begin(), std::negate<int>());
// neg: {-9, -6, -5, -4, -3, -2, -1, -1}
```

람다 [](int x){ return x % 2 == 0; } 는 컴파일러가 내부적으로
functor 클래스를 자동 생성한 것이다. 이름이 필요하거나 여러 곳에서
재사용할 때는 functor가 유리하다.

표준 Functor 목록

비교

Functor	동작
<code>std::greater<T></code>	<code>a > b</code> — 내림차순
<code>std::less<T></code>	<code>a < b</code> — 오름차순 (기본)

산술 / 논리

Functor	동작
<code>std::plus<T></code>	<code>a + b</code>
<code>std::minus<T></code>	<code>a - b</code>
<code>std::multiplies<T></code>	<code>a * b</code>
<code>std::negate<T></code>	<code>-a</code>
<code>std::logical_not<T></code>	<code>!a</code>

람다 — 문법과 알고리즘 활용

STL 알고리즘은 동작을 인자로 받는다. 람다는 그 동작을 코드 안에서 바로 정의하는 이름 없는 함수다.

기본 문법

```
[캡처](매개변수) -> 반환타입 { 본문 }
  ↑           ↑           ↑           ↑
외부변수   함수인자   생략가능   실행코드

[](int x){ return x * 2; }           // 캡처 없음, 반환 타입 자동 추론

[](int a, int b){ return a > b; }    // 매개변수 두 개 - sort의 비교 함수

[](int x) -> bool { return x > 0; } // 반환 타입 명시

[]{ return 42; }                    // 매개변수 없음 - generate용
```

알고리즘별 람다 형태

```
// 단항 조건 (T) → bool   find_if, count_if, copy_if
[](int x){ return x > 0; }

// 이항 비교 (T, T) → bool sort, max_element
[](int a, int b){ return a > b; }

// 단항 변환 (T) → U      transform
[](int x){ return x * 2; }

// 이항 집계 (acc, T) → acc accumulate
[](int acc, int x){ return acc + x; }
```

알고리즘에 바로 전달

```
std::vector<int> v = {3, 1, 4, 1, 5, 9};

// 단항 조건 - find_if
auto it = std::find_if(v.begin(), v.end(),
    [](int x){ return x > 4; });
// *it == 5

// 이항 비교 - sort
std::sort(v.begin(), v.end(),
    [](int a, int b){ return a > b; });
// v: {9, 5, 4, 3, 1, 1}

// 단항 변환 - transform
std::vector<int> sq(v.size());
std::transform(v.begin(), v.end(), sq.begin(),
    [](int x){ return x * x; });
// sq: {81, 25, 16, 9, 1, 1}

// 이항 집계 - accumulate
int sum = std::accumulate(v.begin(), v.end(), 0,
    [](int acc, int x){ return acc + x; });
// sum == 23
```

람다 — 캡처 리스트

람다 바깥의 변수를 본문에서 쓰려면 `[]` 안에 캡처를 선언해야 한다.

값 캡처 `[변수]` — 복사본 사용

```
int threshold = 30;
std::vector<int> v = {10, 20, 30, 40, 50};

// threshold를 값으로 캡처 - 복사본
auto it = std::find_if(v.begin(), v.end(),
    [threshold](int x){ return x > threshold; });
// *it == 40

threshold = 100; // 나중에 바뀌도 람다에 영향 없음
// 람다는 캡처 시점(생성 시)의 30을 기억
```

참조 캡처 `[&변수]` — 원본에 직접 접근

```
int cnt = 0;
std::vector<int> v = {10, 20, 30, 40, 50};

// cnt를 참조로 캡처 - 원본 수정 가능
std::for_each(v.begin(), v.end(),
    [&cnt](int x){ if (x > 25) cnt++; });
// cnt == 3 (30, 40, 50이 조건 충족)
```

캡처 방식 정리

```
int a = 1, b = 2;

[]{} // 캡처 없음 - 외부 변수 사용 불가
[a]{} // a만 값으로 캡처
[&a]{} // a만 참조로 캡처
[a, &b]{} // a는 값, b는 참조
[=]{} // 모든 외부 변수를 값으로 캡처
[&]{} // 모든 외부 변수를 참조로 캡처
[=, &b]{} // 기본 값 캡처, b만 참조로
[&, a]{} // 기본 참조 캡처, a만 값으로
```

캡처	의미	외부 변수 수정
<code>[]</code>	없음	불가
<code>[x]</code>	x 값 복사	불가 (복사본)
<code>[&x]</code>	x 참조	✅ 가능
<code>[=]</code>	전체 값 복사	불가
<code>[&]</code>	전체 참조	✅ 가능

검색 알고리즘

find / find_if

```
std::vector<int> v = {10, 20, 30, 40, 50};

// find: 값으로 검색 - 처음 찾은 이터레이터 반환
auto it = std::find(v.begin(), v.end(), 30);
if (it != v.end()) {
    std::cout << *it;           // 30
    std::cout << (it - v.begin()); // 2 (인덱스)
}

// find_if: 조건으로 검색
auto it2 = std::find_if(v.begin(), v.end(),
    [](int x){ return x > 25; });
// *it2 == 30 (25보다 큰 첫 원소)

// find_if_not: 조건을 만족하지 않는 첫 원소
auto it3 = std::find_if_not(v.begin(), v.end(),
    [](int x){ return x < 40; });
// *it3 == 40
```

count / count_if / any_of / all_of

```
std::vector<int> v = {1, 3, 5, 2, 4, 3, 3};

// count: 특정 값의 개수
std::count(v.begin(), v.end(), 3);    // 3

// count_if: 조건을 만족하는 개수
std::count_if(v.begin(), v.end(),
    [](int x){ return x % 2 == 0; }); // 2 (짝수 개수)

// any_of: 하나라도 조건 만족하는가?
std::any_of(v.begin(), v.end(),
    [](int x){ return x > 4; });      // true (5 있음)

// all_of: 모두 조건 만족하는가?
std::all_of(v.begin(), v.end(),
    [](int x){ return x > 0; });      // true

// none_of: 아무도 조건 만족 안 하는가?
std::none_of(v.begin(), v.end(),
    [](int x){ return x > 10; });     // true
```

정렬 알고리즘

sort / stable_sort

```
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

// 오름차순 정렬 (기본)
std::sort(v.begin(), v.end());
// v: {1, 1, 2, 3, 4, 5, 6, 9}

// 내림차순 정렬 - 비교 함수 전달
std::sort(v.begin(), v.end(), std::greater<int>());
// v: {9, 6, 5, 4, 3, 2, 1, 1}

// 람다로 커스텀 정렬
std::sort(v.begin(), v.end(),
    [](int a, int b){ return a > b; });

// stable_sort: 동일 원소의 상대 순서 유지
struct Student { std::string name; int score; };
std::vector<Student> students = { {"A",90}, {"B",90}, {"C",80} };
std::stable_sort(students.begin(), students.end(),
    [](const Student& a, const Student& b){
        return a.score > b.score;
    });
// score 같으면 기존 순서 유지: A, B, C
```

partial_sort / nth_element

```
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

// partial_sort: 상위 N개만 정렬
std::partial_sort(v.begin(), v.begin() + 3, v.end());
// 처음 3개만 정렬: {1, 1, 2, ...나머지 순서 불보장}

// nth_element: N번째 원소를 제자리로
std::vector<int> v2 = {3, 1, 4, 1, 5, 9, 2, 6};
std::nth_element(v2.begin(), v2.begin() + 3, v2.end());
// v2[3]에는 정렬 시 4번째 원소가 위치
// 앞쪽은 v2[3]보다 작거나 같고, 뒤쪽은 크거나 같음
```

is_sorted 확인

```
std::vector<int> a = {1, 2, 3, 4, 5};
std::vector<int> b = {3, 1, 2};

std::is_sorted(a.begin(), a.end()); // true
std::is_sorted(b.begin(), b.end()); // false
```


변환 알고리즘

transform

```
#include <algorithm>

std::vector<int> v = {1, 2, 3, 4, 5};
std::vector<int> result(v.size());

// 각 원소에 함수 적용 → result에 저장
std::transform(v.begin(), v.end(),
               result.begin(),
               [](int x){ return x * x; });
// result: {1, 4, 9, 16, 25}

// 제자리 변환 (in-place)
std::transform(v.begin(), v.end(),
               v.begin(),
               [](int x){ return x * 2; });
// v: {2, 4, 6, 8, 10}

// 두 범위를 합쳐 변환
std::vector<int> a = {1, 2, 3};
std::vector<int> b = {4, 5, 6};
std::vector<int> c(3);
std::transform(a.begin(), a.end(),
               b.begin(), c.begin(),
               [](int x, int y){ return x + y; });
// c: {5, 7, 9}
```

copy / copy_if / fill / generate

```
std::vector<int> src = {1, 2, 3, 4, 5};
std::vector<int> dst(5);

// copy: 범위를 복사
std::copy(src.begin(), src.end(), dst.begin());
// dst: {1, 2, 3, 4, 5}

// copy_if: 조건을 만족하는 원소만 복사
std::vector<int> evens;
std::copy_if(src.begin(), src.end(),
             std::back_inserter(evens),
             [](int x){ return x % 2 == 0; });
// evens: {2, 4}

// fill: 모든 원소를 같은 값으로
std::fill(dst.begin(), dst.end(), 0);
// dst: {0, 0, 0, 0, 0}

// generate: 함수 호출 결과로 채우기
int n = 0;
std::generate(dst.begin(), dst.end(),
              [&n]{ return n++; });
// dst: {0, 1, 2, 3, 4}
```

remove / unique — 제거와 중복 처리

remove / remove_if — erase-remove 패턴

```
// ⚠ std::remove는 실제로 삭제하지 않는다!  
// 제거 대상을 뒤로 밀고, 유효 범위의 끝을 반환한다.  
  
std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};  
  
auto new_end = std::remove(v.begin(), v.end(), 2);  
// v: {1, 3, 4, 5, ?, ?, ?} ← ? 는 쓰레기값  
// new_end → 유효 원소 다음을 가리킴  
  
// erase로 실제 삭제 — erase-remove 관용구  
v.erase(new_end, v.end());  
// v: {1, 3, 4, 5}  
  
// 한 줄로 (관용구 형태)  
v.erase(std::remove(v.begin(), v.end(), 2), v.end());  
  
// remove_if: 조건으로 제거  
v.erase(  
    std::remove_if(v.begin(), v.end(),  
        [](int x){ return x % 2 == 0; }),  
    v.end());  
// v: {1, 3, 5} (짝수 모두 제거)
```

unique — 연속 중복 제거

```
// ⚠ unique도 실제로 삭제하지 않는다.  
// 연속된 중복을 뒤로 밀고 유효 범위의 끝을 반환한다.  
  
// 연속된 중복만 제거 (정렬 없이)  
std::vector<int> v1 = {1, 1, 2, 3, 3, 3, 4};  
v1.erase(std::unique(v1.begin(), v1.end()), v1.end());  
// v1: {1, 2, 3, 4}  
  
// 모든 중복 제거: sort 후 unique  
std::vector<int> v2 = {3, 1, 4, 1, 5, 9, 2, 6, 5};  
std::sort(v2.begin(), v2.end());  
// v2: {1, 1, 2, 3, 4, 5, 5, 6, 9}  
v2.erase(std::unique(v2.begin(), v2.end()), v2.end());  
// v2: {1, 2, 3, 4, 5, 6, 9}
```

remove / unique 는 원소를 물리적으로 삭제하지 않는다. 반드시
erase 와 짝지어 사용해야 컨테이너 크기가 줄어든다.

for_each / reverse / binary_search

for_each — 각 원소에 동작 적용

```
std::vector<int> v = {1, 2, 3, 4, 5};

// 읽기 — 각 원소 출력
std::for_each(v.begin(), v.end(),
    [](int x){ std::cout << x << " "; });
// 1 2 3 4 5

// 쓰기 — 참조로 캡처해 원본 수정
std::for_each(v.begin(), v.end(),
    [](int& x){ x *= 2; });
// v: {2, 4, 6, 8, 10}
```

reverse — 순서 뒤집기

```
std::vector<int> v = {1, 2, 3, 4, 5};
std::reverse(v.begin(), v.end());
// v: {5, 4, 3, 2, 1}

std::string s = "hello";
std::reverse(s.begin(), s.end());
// s: "olleh"
```

binary_search / lower_bound — 정렬된 범위 검색

```
std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9};
// ⚠ 반드시 정렬된 상태여야 한다

// binary_search: 존재 여부 확인 — bool 반환
std::binary_search(v.begin(), v.end(), 5); // true
std::binary_search(v.begin(), v.end(), 99); // false

// lower_bound: key 이상인 첫 이터레이터
auto lb = std::lower_bound(v.begin(), v.end(), 5);
// *lb == 5

// upper_bound: key 초과인 첫 이터레이터
auto ub = std::upper_bound(v.begin(), v.end(), 5);
// *ub == 6

// 값이 없을 때 — end() 반환
auto it = std::lower_bound(v.begin(), v.end(), 99);
if (it == v.end()) std::cout << "없음\n";
```

binary_search / lower_bound / upper_bound 는 정렬된 범위에서만 올바르게 동작한다. $O(\log n)$.

집계 알고리즘

accumulate

```
#include <numeric>

std::vector<int> v = {1, 2, 3, 4, 5};

// 합계 (초기값 0)
int sum = std::accumulate(v.begin(), v.end(), 0);
// sum == 15

// 곱 (초기값 1)
int product = std::accumulate(v.begin(), v.end(), 1,
    [](int acc, int x){ return acc * x; });
// product == 120

// 문자열 연결
std::vector<std::string> words = {"C++", " ", "STL"};
std::string sentence = std::accumulate(
    words.begin(), words.end(), std::string{});
// sentence == "C++ STL"
```

max_element / min_element / minmax_element

```
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

// 최댓값 이터레이터
auto max_it = std::max_element(v.begin(), v.end());
std::cout << *max_it;    // 9
std::cout << (max_it - v.begin()); // 5 (인덱스)

// 최솟값 이터레이터
auto min_it = std::min_element(v.begin(), v.end());
std::cout << *min_it;    // 1

// 최솟값·최댓값 동시에
auto [lo, hi] = std::minmax_element(v.begin(), v.end());
std::cout << *lo << ", " << *hi;    // 1, 9
```

iota — 순차 값 채우기

```
#include <numeric>
std::vector<int> v(5);
std::iota(v.begin(), v.end(), 1);
// v: {1, 2, 3, 4, 5}
```

알고리즘 조합 예제

학생 성적 데이터를 STL 알고리즘으로 처리한다.

알고리즘은 **이터레이터로 컨테이너와 분리되어** 있어 재조합이 자유롭다.

```
#include <vector>
#include <algorithm>
#include <numeric>
#include <iostream>
#include <string>

struct Student { std::string name; int score; };

int main() {
    std::vector<Student> students = {
        {"Alice", 85}, {"Bob", 92}, {"Carol", 78},
        {"Dave", 96}, {"Eve", 70}, {"Frank", 88}
    };

    // ① 점수 내림차순 정렬
    std::sort(students.begin(), students.end(),
        [](const Student& a, const Student& b){
            return a.score > b.score;
        });

    // ② 80점 이상인 학생 수
    int pass = std::count_if(students.begin(), students.end(),
        [](const Student& s){ return s.score >= 80; });
    std::cout << "합격: " << pass << "명\n";
    // 합격: 4명
```

```
// ③ 전체 평균 점수
int total = std::accumulate(students.begin(), students.end(), 0,
    [](int acc, const Student& s){ return acc + s.score; });
double avg = static_cast<double>(total) / students.size();
std::cout << "평균: " << avg << "\n";
// 평균: 84.83...

// ④ 90점 이상 학생 추출
std::vector<Student> top;
std::copy_if(students.begin(), students.end(),
    std::back_inserter(top),
    [](const Student& s){ return s.score >= 90; });
for (const auto& s : top)
    std::cout << s.name << "(" << s.score << ")\n";
// Dave(96) Bob(92)

return 0;
}
```